

Machine Learning - Regression

Lung Cancer Prediction using Multivariate Regression

Aarya Patil - TY CSE - 22070122140
Ayushi Kapgate - TY CSE - 22070122093

Introduction

- **Objective:** To predict the percent of lung cancer, and chance of survival for a patient using multivariate regression.

WHY :-

- *Lung cancer is one of the top causes of death from cancers in the world.*
- *An early prediction based on clinical factors and genomic data will increase survival rates due to timely treatment.*
- *Medical predictions are widely performed with the help of special machine learning techniques, like regression.*
- *We are using Multivariate regression, which involves predicting more than one dependent (output) variables.*

Methodology

- **Data Collection:** *We referenced and used online datasets, which had genuine data regarding clinical symptoms of lung cancer.*
- **Data Preprocessing:** *Some of the data was irrelevant and inconsistent, it was changed so to apply multivariate regression. For every attribute, the extent to which patients are experiencing the symptoms is represented on the scale of 10.*
- **Selection of Model:** *Multivariate regression is selected on the grounds that it can analyze multiple variables, to predict more than one outputs.*
- **Tools:** *Python language, and Jupyter Notebook implemented in VS Code IDE for Development and Testing.*

MODEL TRAINING →

Model Training

1. Import required python libraries; pandas, numpy, sci-kit learn, etc.
2. Load the dataset using **pandas** library.
3. Select attributes which are irrelevant, to be dropped.
4. Select and process input variables and output variables ('OUTPUT', SURVIVAL_PROBABILITY) from the dataset.
5. Split the data into training and testing data using **train_test_split()** function.
- 6.. Here training data is 80% and testing data is 20%.
7. Train the model using **LinearRegression()** function, and predict the output.
8. Scaling method is also applied to scale each attribute to the same scale with respect to each other.
9. Model Evaluation:
 - a. Mean Square Error (MSE)
 - b. R^2 score
 - *Lower value for MSE and a Higher value for R^2 indicates model is trained well, and output is accurate upto a good extent.*

Results and Discussion

- a. **Mean Square Error:** MSE measures the average squared difference between the actual values and the predicted values.
The Lower value obtained indicates the difference in actual and predicted values is less, hence model prediction is efficient.
- b. **R^2 score:** Measures the proportion of variance in the dependent (target) variable that is predictable from the independent variables. A value closer to 1 indicates model prediction is good.

Conclusion

- The acquired values of parameters of '**OUTPUT**' is;

MSE: 0.02040154753158398

R^2 : 1.0

And parameters of '**SURVIVAL PERCENTAGE**' is;

MSE: 97.95616377661568

R^2 : -0.06006884621798969

shows that model is trained well and is predicting efficiently.

- We can also use other models like Ridge Regression, or LASSO Regression, to predict the same.

Literature Review

- Fallowfield L, Gagnon D, Zagari M, Cella D, Bresnahan B, Littlewood TJ, McNulty P, Gorzegno G, Freund M; Epoetin Alfa Study Group. Multivariate regression analyses of data from a randomised, double-blind, placebo-controlled study confirm quality of life benefit of epoetin alfa in patients receiving non-platinum chemotherapy. Br J Cancer. 2002 Dec 2;87(12):1341-53. doi: 10.1038/sj.bjc.6600657. PMID: 12454760; PMCID: PMC2376290.
- K. Sivanagireddy, S. Yerram, S. S. N. Kowsalya, S. S. Sivasankari, J. Surendiran and R. G. Vidhya, "Early Lung Cancer Prediction using Correlation and Regression," 2022 International Conference on Computer, Power and Communications (ICCPC), Chennai, India, 2022, pp. 24-28, doi: 10.1109/ICCPC55978.2022.10072059. keywords: {Machine learning algorithms;Correlation;Computational modeling;Linear regression;Lung cancer;Lung;Machine learning;Lung cancer;machine learning;Regression Model;Small cell lung cancer},
- <https://anson.ucdavis.edu/~jie/remMap.pdf>
- <https://www.sciencedirect.com/science/article/pii/S0929664611000696>

References

- <https://www.kaggle.com/datasets/yusufdede/lung-cancer-dataset>
- <https://portal.gdc.cancer.gov/>
- <https://www.kaggle.com/datasets/mysarahmadbhat/lung-cancer>
- <https://www.mygreatlearning.com/blog/introduction-to-multivariate-regression/>

Code

```
# import libraries
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import PolynomialFeatures

# read data from csv file
data = pd.read_csv(r"C:\Users\aaarya\Desktop\Specialization
AIML\Projects\lung_cancer.csv")
```

#function to get shape of the dataset

```
def get_shape(df):
```

```
    print('there are', df.shape[0], 'rows and', df.shape[1], 'columns.')
```

```
get_shape(data)
```

printing some rows of the dataset

```
data.head()
```

choose input features. Features having low correlation values are removed.

```
X = data.drop(['AGE', 'OUTPUT', 'GENDER', 'COUGHING', 'SWALLOWING DIFFICULTY',  
'LUNG_CANCER', 'SURVIVAL_PERCENTAGE'], axis=1)
```

choose output features

```
y = data[['OUTPUT', 'SURVIVAL_PERCENTAGE']]
```

splitting the data into training (80%) and testing data (20%)

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
# scaler function to scale down every attribute to the same scale.  
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

```
# linear regression model training using training data.  
model = LinearRegression()  
model.fit(X_train_scaled, y_train)
```

```
# model testing using testing data.  
y_pred = model.predict(X_test_scaled)
```

```
# Mean Square Error and R2 score for attribute 'OUTPUT'.  
mse_output = mean_squared_error(y_test['OUTPUT'], y_pred[:, 0])  
r2_output = r2_score(y_test['OUTPUT'], y_pred[:, 0])  
print("MSE for 'OUTPUT': ", mse_output)  
print("R2 for 'OUTPUT': ", r2_output)
```

```
# Mean Square Error and R2 score for attribute 'SURVIVAL_PROBABILITY'.
mse_survival = mean_squared_error(y_test['SURVIVAL_PERCENTAGE'], y_pred[:, 1])
r2_survival = r2_score(y_test['SURVIVAL_PERCENTAGE'], y_pred[:, 1])
print("MSE for 'SURVIVAL_PERCENTAGE': ", mse_survival)
print("R2 for 'SURVIVAL_PERCENTAGE': ", r2_survival)
```

Predicting output for custom inputs.

```
sample_input = np.array([
    10.0000000, 7.586233, 9.586483, 10.0000000, 8.096048, 5.285471,
    6.285662, 5.341830, 9.986573, 9.857234, 5.323830, 9.754613,
    8.892104, 0.456789]
])
sample_input_scaled = scaler.transform(sample_input)
sample_prediction = model.predict(sample_input_scaled)
print("Cancer detected on the scale of 10, and Survival Percentage is: ",
sample_prediction)
```

Output

```
... Cancer detected on the scale of 10, and Survival Percentage is: [[ 7.82528855 66.44356976]]  
c:\Users\arya\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\base.py:493: UserWarning: X does not  
warnings.warn(
```

Thank You!